

Shorts 1: API calls

use the "requests" module to send requests over the internet to a certain API of my choice.

Endpoints are used to access the API from a certain domain. Use a "get" request to access whatever data the API has in a particular context.

ex. the Chicago Art Museum (api.py)

```
import requests

response = requests.get("https://api.artic.edu/api/v1/artworks/search")
print(response)
```

Annotations for the code above:

- `import requests`: the route to the API
- `"https://api.artic.edu/api/v1/artworks/search"`: the specific domain we are looking for

terminal:

```
>>> python api.py
<Response [200]>
```

"200" refers to the HTTP status code, meaning everything went ok

The Art Institute acknowledged our request to search some of their artwork.

Essentially, an API is the user sending a request to a certain 3rd party service, and that service responding back with some info that might have been requested, allowing the user to access their data and include it in the user's program without having to download anything. Directly search it, etc.

To actually read the response, the user must convert the response into a JSON file.

ex. using "requests"

```
response = requests.get("____URL____")
file = response.json()
```

Annotations for the code above:

- `"____URL____"`: Get the data
- `response.json()`: Convert to JSON

JSON files retrieved from an API usually have a documentation that specifies what keys are associated with what values.

To retrieve the values/keys of JSON files:

ex. for dataset in file["data"]:
 print(dataset["more data"])

for each key from file["data"] where I defined the keys as "dataset", I get more information from dataset by getting information from within the key "more data".

If I am retrieving information online / query APIs that I find online, I need to have checks to make sure the request is working as intended.

ex. Using exceptions:

```
try:  
    response = requests.get("____URL____")  
except requests.HTTPError:  
    print("pomp!")  
    return
```

try to get the response from the specified URL

error with the HTTP request

In the requests library documentation, one can call `.raise_for_status()`. What this does is check if the response actually worked as intended, meaning "<Response [200]>" does not show "200".

Using it: `response.raise_for_status()`

Certain APIs can be parameterized, according to their documentation.

ex using parameters from Chicago Art Museum:

```
response = requests.get(api.py  
    "https://api.artic.edu/api/v1/artworks/search",  
    {"q": "Monet"} ← use a dictionary to specify parameter and value  
)
```

terminal

```
>>> python api.py  
* <artwork1 by Monet>  
* <artwork2 by Monet>  
* <artwork3 by Monet>  
.  
.  
.  
* <artwork n by Monet>
```

displays the names of the artworks
by the artist Monet

You can use the input function to ask the user for input, and use the value of that user input by setting it as the value of the parameter.

Shorts 2: Creating Modules and Packages

Create a new file with some function that will be reused throughout different files.

ex. hello.py

```
def hi(name):  
    return f"yo {name}"
```

call.py

```
import hello
```

```
def main():
```

```
    leName = input("your name: ")  
    poop = hello.hi(leName)  
    print(poop)
```

terminal

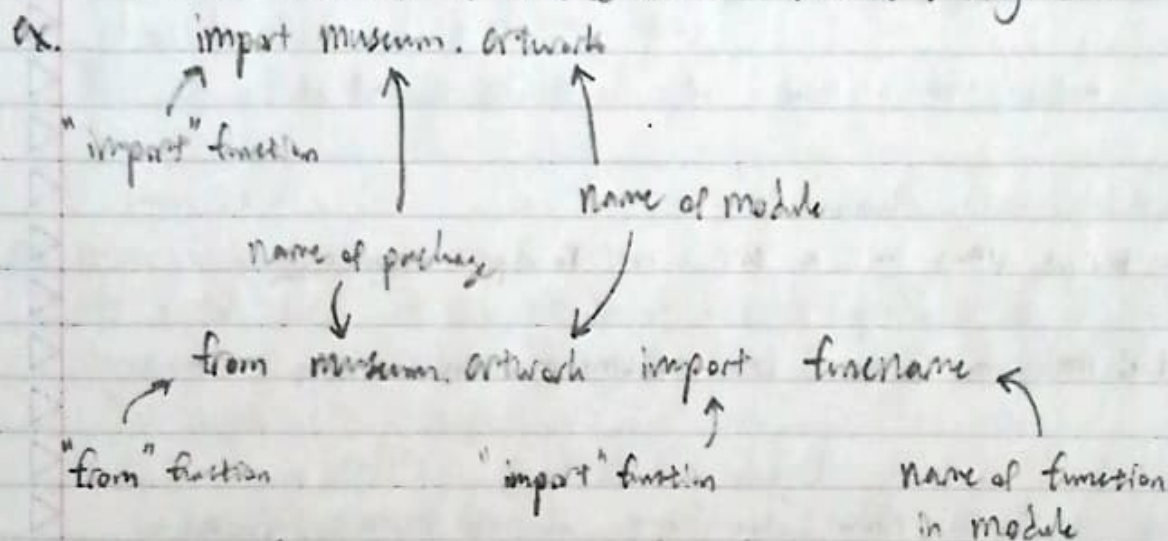
```
your name: pooni's  
yo pooni's
```

The user can create a Package in python.

Package: a collection of multiple modules, created by making a folder.

1. make a folder to hold the different modules
2. put desired modules into the appropriate folder
3. add a special file, `"__init__.py"`
 - the special file does not need to have anything in it. It simply helps Python realize that a certain folder is a package with multiple modules

To use a package: Utilize the same functions used on modules, but use the dot notation to indicate which file/module is being used.



Shorts 3: Random

- ① `Choice()`: takes some input as a list and returns some random value from that list
- ② `Choices()`: returns multiple items from a given list; also takes another argument (`k`), which is the number of items randomly chosen from the list
 - There can be duplicate choices. This is called Sampling with Replacement:
Shuffle, choose, return, put back, Shuffle, choose...

- ⊕ . Sample(): takes 2 arguments, one being a list of values and the other being "k", the number of items to select from the list.
- Unlike . Choices(), . Sample() will Select Without Replacement.

There is another argument in the . choices() function called "weights", that takes in a list of number values. Weights shows the probability of one thing to happen more likely than another.

ex. Using Weights

```
import random  
num = [1, 2, 3]
```

```
print(random.choices(num, weights = [100, 0, 0], k=2))
```

100% chance to get '1', 0% chance
↓ for '2' and '3'

Weights should be numerical values that add up to 100, with the same number of values as the chosen list, to represent percentages.

It is important to delay with randomness by guaranteeing certain outcomes when we need to debug.

There is a function called Seed that goes into some pseudorandom number generator. By giving a seed a specific number that stays the same, we will always get the same choices.

- ⊕ . Seed(): changes on each run of the program, but can be defined explicitly for every time the program is run.
- Setting the seed still gives a random choice/solution, but it will stay consistent for every run.

Shorts 4: Style

Style is subjective and depends on the programmer, the company, the course, and the language.

In Python: PEP 8 - Python Enhanced Proposal, a set of proposals that the community of Python generated to codify standards for the sake of readability and maintainability.

- Indentation
- Tabs or Spaces
- Maximum Line Length
- Blank lines
- Imports
- more rules...

A tool to solve formatting problems is with a tool called "pylint"
pylint: a linter, a program that statically analyzes code and tries to figure out any mistakes and inconsistencies with style

Install with: `pip install pylint`

Another tool/alternative is "pycodestyle"

pycodestyle: less noisy and annoying than pylint, as it will take care of the reformating process (basically autocorrect)

Another alternative is "black"

black: a more opinionated formatting program for companies and different courses

installing: `pip install black`

running: (in terminal window) `>>> black filename.py`